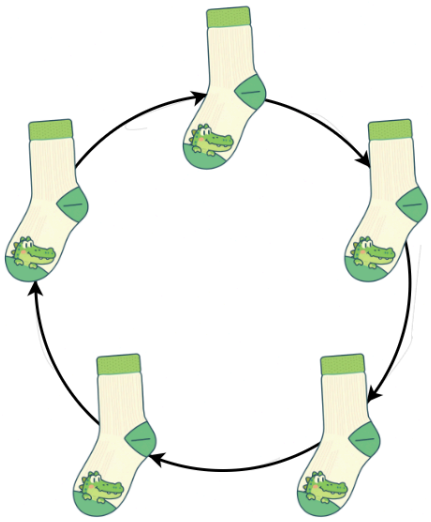
Department of Information Technology and Electrical Engineering
Integrated Systems Laboratory
Spring Semester 26

Improving Serial Link and Enabling a Ring Bus Topology

Semester Project Report



Llorenç Muela Hausmann, Fabian Aegerter
lmuela@ethz.ch, faegerter@ethz.ch

June 05, 2026

Advisors:
- Pu Deng, pudeng@iis.ee.ethz.ch
- Tim Fischer, fischeti@iis.ee.ethz.ch

Professor:
- Prof. Dr. L. Benini, lbenini@iis.ee.ethz.ch

Acknowledgements

We would like to thank our supervisors, Pu Deng and Tim Fischer, for their presence during the weekly meetings and for the discussions that helped accompany the progress of this project.

We also thank Professor Luca Benini for providing the academic supervision and framework for the project.

Abstract

The Serial Link IP developed at the Integrated Systems Laboratory (IIS) of ETH Zürich provides a point-to-point bridge that joins the AXI domains of two SoCs over a small number of serial wires.

This work extends that IP so that a SoC becomes a node in a unidirectional ring interconnect, allowing more than two SoCs to communicate over a simple, low-pin-count topology. To this end, the host-facing protocol is migrated from AXI to the lightweight OBI bus used by the target platforms, and the upper layers are extended with networking logic: address-based routing, in-order response delivery through a zero-latency reorder buffer, and detection of undeliverable payloads. Two optimizations reduce the cost of each hop: dynamic payload sizing transmits only as many cycles as a given transaction type requires, and a protocol-layer bypass forwards transit traffic without reassembly. Credit-based flow control is adapted to the topology through a counter-rotating credit-return clock.

The resulting IP is integrated into the Croc SoC, verified in simulation, and successfully placed and routed in a 130nm technology, and its performance is characterized across several ring-load scenarios.

Declaration of Originality

We hereby declare that we authored the work in question independently. In doing so we only used the authorized aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor. For a detailed version of the declaration of originality, please refer to Section B.

List of Acronyms

AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
CDC	Clock Domain Crossing
DDR	Double Data Rate
FIFO	First In, First Out
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GE	Gate Equivalent
HDL	Hardware Description Language
IC	Integrated Circuit
IIS	Integrated Systems Laboratory
IP	Integrated Peripheral
MSB	Most Significant Bit
OBI	Open Bus Interface
PDK	Process Design Kit
RTL	Register-Transfer Level
RX	Receiver
SRAM	Static Random-Access Memory
SoC	System-on-Chip
TX	Transmitter

Contents

List of Figures

List of Tables

1

Introduction

1.1 Motivation

Building a single SoC large enough to hold all the compute a workload needs is increasingly impractical. A common alternative is to split the work across several smaller SoCs and connect them together. The difficulty is the connection itself: off-chip wires are expensive, and the number of pins a chip can dedicate to communication is limited.

The Serial Link IP developed at the Integrated Systems Laboratory (IIS) addresses this by serializing a wide Advanced eXtensible Interface (AXI) bus onto a small number of wires, letting two SoCs behave as if they shared one bus. It is, however, a strictly point-to-point bridge: it joins exactly two SoCs. Connecting more than two with point-to-point links does not scale well, since linking every node to every other node multiplies both the number of links and the pin count.

A ring topology avoids this. Each node connects only to its two neighbours, so the pin count per node stays constant no matter how many SoCs join the network, and the routing stays simple. The price is that data may have to hop through intermediate nodes to reach its destination. Given how few wires a ring needs, this trade-off is attractive for connecting several SoCs.

This project therefore extends the existing Serial Link IP so that an SoC becomes a node in a unidirectional ring, rather than rebuilding such a link from scratch. At the same time, the host-facing bus is migrated from AXI to the lighter OBI protocol used by the target platforms, so that the IP fits naturally into SoCs such as Croc.

1.2 Contributions

This work turns the point-to-point Serial Link IP into a ring node. The main contributions are:

- **Protocol migration:** the host interface is moved from AXI to OBI, and the configuration registers are moved from Advanced Peripheral Bus (APB) to OBI, so the IP presents a single, consistent bus to the host.
- **Protocol layer:** address-based routing derives a destination node from the upper address bits, requests and responses are classified and forwarded around the ring, responses are delivered to the host in order through a zero-latency reorder buffer, and undeliverable payloads are detected through loop-back handling.

- **Dynamic payload sizing:** each of the four transaction types is sent at its own size, so small transactions no longer pay the cost of the largest payload.
- **Protocol-layer bypass:** transit traffic is forwarded directly from the receive side to the transmit side without reassembly, saving several cycles per hop on a lightly loaded ring.
- **Counter-rotating credit return:** credit-based flow control is adapted to the unidirectional ring by returning credits as clock pulses on a single wire running opposite to the data.
- **Integration and evaluation:** the IP is integrated into the Croc SoC, verified in simulation, placed and routed in a 130nm technology, and characterized across several ring-load scenarios.

1.3 Outline

Section 2 introduces the two concepts the rest of the report relies on: the ring network topology and the OBI protocol. Section 3 describes the original point-to-point Serial Link IP that this work builds upon, focusing on its datapaths, payloads, and credit system. Section 4 presents the ring Serial Link itself, following the module from the network-level decisions of payloads and routing down through the implemented protocol, data link, and physical layers, and covering the configuration registers and the removal of the power-management wrapper. Section 5 describes the integration into the Croc SoC, the software testing, and the backend flow. Section 6 reports the simulation results and analyses how the design scales with ring size, payload sizing, the bypass, the number of lanes, and the number of outstanding transactions. Section 7 summarizes the work and outlines directions for future work.

2

Background

2.1 Ring Network

A ring network is a topology in which each node is connected to exactly two others, such that the network forms a closed loop. Data can traverse the ring either unidirectionally or bidirectionally depending on the configuration. The main advantages of this topology are its simple routing and compact size relative to more complex alternatives. The trade-off is that data cannot reach its destination directly — it must hop through intermediate nodes along the ring.

2.2 OBI Protocol

The OBI protocol is a synchronous on-chip bus interface designed for efficient communication between components in SoCs. Its goal is to provide a simple and efficient communication protocol that also enables easier interfacing with Advanced Microcontroller Bus Architecture (AMBA) protocols such as AXI [2]. In practice, the OBI protocol is commonly implemented with a crossbar, which allows any manager to communicate with any subordinate as long as neither is currently occupied with a transaction. This simplicity makes it a popular choice for the data bus in SoCs.

The OBI protocol supports many optional signals. The following describes the channel structure and the most important signals:

Address Channel: Used by the manager to issue transaction requests, including the target address, transaction type, and write data [2].

- **Request Signal (req):** Indicates the initiation of a transaction request and the validity of the manager address channel signals.
- **Grant Signal (gnt):** Handshake signal from the subordinate acknowledging and granting the processing of the transaction request.
- **Address Signal (addr):** Specifies the target address for the transaction.
- **Write Enable Signal (we):** Specifies the transaction type — high for a write operation, low for a read.
- **Write Data Signal (wdata):** Carries the data to be written during a write transaction.
- **Byte Enable Signal (be):** Specifies which bytes within the write data are valid.
- **Address Channel Transaction Identifier (aid):** Carries the transaction ID, allowing responses to be matched to their corresponding requests.

Response Channel: Used by the subordinate to issue the responses such as sending requested data and acknowledgments to the manager.

- **Response Valid Signal (rvalid):** Indicates that the subordinate has valid signals ready to be transmitted.
- **Read Data Signal (rdata):** Transfers the read data from the subordinate to the manager.
- **Response Channel Transaction Identifier (rid):** Carries the transaction ID of the corresponding request.

3

The Original Serial Link

The original Serial Link IP [3] was developed at the IIS at ETH Zürich for the PULP platform [4]. It enables peer-to-peer communication between two SoCs, connecting two AXI protocol domains through multiple link channels using the AXI stream protocol. This allows the two SoCs to act as if they shared the same AXI domain, with the Serial Link IP serving as a bridge between them. The most noteworthy aspects of the original design are the structuring of the RX and TX datapaths — including the ability to split payloads across multiple link channels, the payload structure and credit system in general.

3.1 Channels, Lanes, and Double Data Rate

The RX datapath of the original Serial Link IP is depicted in Figure 1. The figure shows the IP using the Double Data Rate (DDR) functionality with a single AXI stream link channel. The Serial Link IP allows the number of bit lanes to be configured which, together with the clock, form a link channel for data transmission — i.e., a single serial link channel can have a variable bit width. In addition, the number of serial link channels can be scaled up. The module handles multiple serial link channels by splitting a single payload across them and reassembling it on the receiving end, while also allowing non-functional serial link channels to be disabled in order to improve reliability. This enables higher throughput when additional pins are available for more link channels and lanes. The examples in Figure 1 and Figure 2 show only a single serial link channel with a bit lane width of 8. Beyond link channels and lanes, DDR functionality can be enabled, allowing data to be sampled on both the rising and falling edges of the clock during transmission between the two SoCs. This reduces the required clock frequency by a factor of two. Additionally, the outgoing clock in the ring is phase shifted by 90 degrees in order to sample the data at the middle of its period, which significantly increases the chance that the data is sampled correctly and softens the race condition between data and clock at the receiving serial link.

As shown in Figure 1, the RX datapath consists of flip-flops that buffer the incoming DDR signal in order to feed the full number of bits per cycle into a Clock Domain Crossing (CDC) First In, First Out (FIFO) Gray buffer. This enables the transition from the AXI stream clock domain of the sending SoC into the clock domain of the receiving SoC. Depending on the number of serial link channels, this portion of the RX datapath is replicated in parallel, with one instance per serial link channel. The data from one or more CDC FIFO Gray buffers is then fed into the control flow FIFO buffer. To reassemble

the payload in parallel, 16-bit slices are forwarded into the stream registers, which — once all are filled — trigger the signal that unpacks the payload and initiates a standard AXI request or response, depending on the payload type.

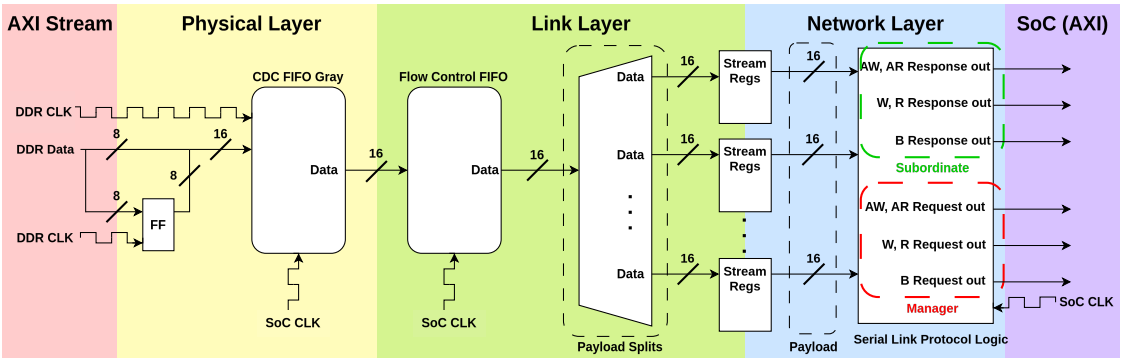


Figure 1: Block Diagram of one Channel in the original Serial Link RX Datapath

The TX datapath of the original Serial Link IP performs the inverse of the RX datapath. It accepts AXI requests from the SoC AXI domain, packs them into a payload, and stores the payload in the stream FIFO. The payload is then serialized into 16-bit slices, of which 8 bits are buffered in a flip-flop to enable DDR transmission across the 8-bit lanes. A notable aspect of the TX datapath is the DDR clock generation: the Serial Link IP allows the outgoing clock frequency to be configured and actively adjusted by the SoC domain through the clock frequency division setting in the IP's configuration registers.

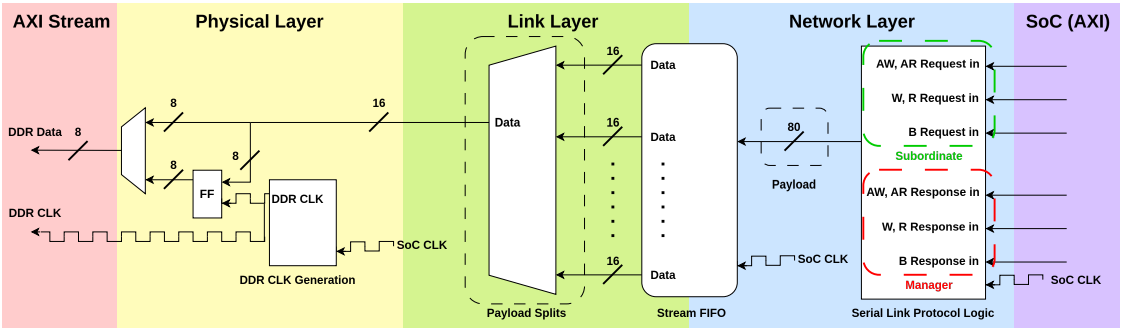


Figure 2: Block Diagram of one Channel in the original Serial Link TX Datapath

3.2 Payloads and Credit Based Flow Control

One potential issue in the Serial Link IP is an overflow of the control flow FIFO buffer in the RX datapath shown in Figure 1. To prevent this, the Serial Link IP introduces a credit system in which the credit count is initialized to the number of payloads the control flow FIFO buffer can hold. Each time a SoC sends a payload, the credit count is decremented by one. When the count reaches zero, the Serial Link IP stops accepting new requests from the SoC, preventing the control flow FIFO of the other SoC from overflowing. The two Serial Link IPs keep track of how many credits they have to send back and how many credits they still have available to send payloads.

Payloads in the Serial Link IP contain the data and its packaging transmitted between two Serial Link modules. Each payload consists of several fields: a header, an AXI beat,

a B channel valid signal, and returned credits. The header indicates which AXI channel the payload carries (AR, AW, R, or W) or if it represents an idle credit payload. Since a B channel response can always be appended to any payload, its valid signal is included as a dedicated field in every payload. Notably, the write response ready signal (B ready) is not transmitted. B channel responses and credits can be added to any type of payload. When one Serial Link's count of credits that it has to send back grows too high, it sends out an idle payload. Idle payloads only carry B channel responses and credits. Every payload has the same size as the largest possible payload in the original design: a smaller payload is zero-padded to that size, and if the largest payload is not a multiple of the bandwidth in bits, it too is zero-padded so that the transmitted data is always fully defined.

4

The Ring Serial Link

The ring variant turns the one-to-one connection into a node of a unidirectional ring interconnect. Two major changes form the pillars of this new IP. First, the bus seen by the host on the SoC side is migrated from AXI to the lightweight OBI protocol used by the target SoC. Second, the upper layers gain new networking logic: addressing, routing, in-order response delivery, and detection of undeliverable payloads. Furthermore, the ring serial link implementation introduces two main optimizations, dynamic payload sizes and protocol-layer bypass, that keep each hop fast and shrink the per-transaction footprint on the wires.

4.1 Architectural Overview

The ring serial link has been designed to enable SoCs in the network to communicate in a unidirectional ring topology, such that every node forwards data to exactly one successor and receives data from exactly one predecessor. This configuration is illustrated in Figure 3. Because the data travels in a single direction, the credit-based flow control had to be implemented to travel in the opposite direction to make communication and capacity synchronization optimal. This counter-rotating credit path is further discussed in Section 4.4.3.

To send data to, or receive data from, another node in the network, the IP reuses the host's existing address space instead of introducing a separate routing namespace. When an OBI request is issued to the Ring Serial Link IP, the network layer extracts the destination node ID from the upper bits of the request address. This defines a global address format composed of two fields: the upper bits identify the destination node, while the lower bits specify the local address within that destination node.

Off-node accesses are therefore translated by interpreting the upper address bits as the destination node ID. Conversely, when a node receives a request addressed to itself, it removes these upper bits to recover the corresponding local address. Each node's ID is configured at runtime through a dedicated register. The node ID width is controlled by the RDL parameter `Log2MaxNodeIds` (denoted N), enabling rings of configurable size with up to 2^N nodes.

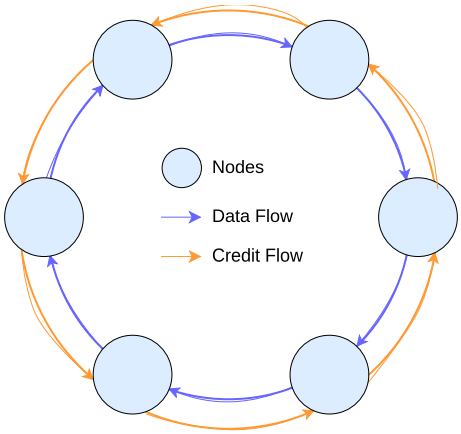


Figure 3: Unidirectional ring of nodes. Payloads circulate in one direction; flow-control credits are returned in the opposite direction.

4.2 Payloads and Routing

Communication between nodes in the network is performed by forwarding payloads from one node to the next. These payloads carry the necessary information to reconstruct the OBI transaction at its destination and replay it to its host.

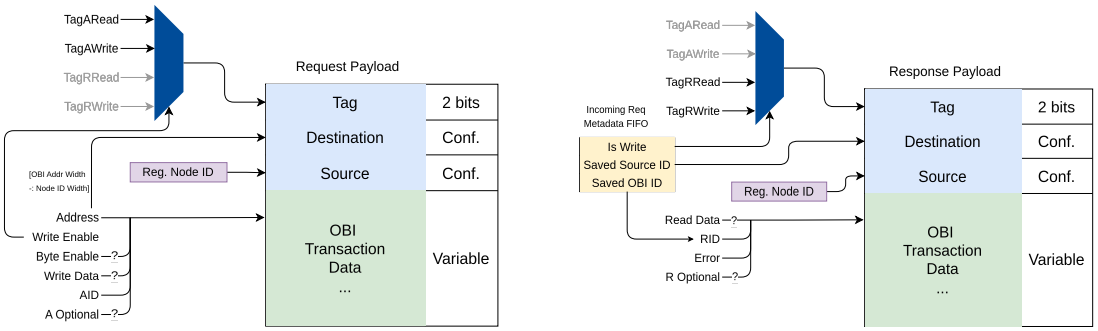
4.2.1 Payload Anatomy

Each payload can be defined as being composed of a head and a body. The head contains the information managed by the IP that is specific to the ring network. This information allows the IP to properly route the payload, process it, and ensure correct operation within the network. The body, on the other hand, contains the OBI bus data after restructuring and serialization, and is used to reconstruct the original transaction at the destination node.

More specifically, the head contains the following fields:

- *Tag*: identifies the transaction type.
- *Source ID*: indicates the node from which the payload originated.
- *Destination ID*: indicates the node to which the payload is meant to be sent.

Figure 4 depicts the structure of a payload, while Figure 4a shows more concretely how the translation of an OBI request is translated into a payload structure and Figure 4b illustrates how an OBI response is translated.



(a) Request Payload

(b) Response Payload

Figure 4: Mapping of data to payload structure.

4.2.2 Transaction Types and the Header Tag

Although the OBI protocol is organized around address and response channels, its transactions and signals can be grouped into four transaction types: write requests, read requests, write responses, and read responses. Additionally, we also observe that even the set of signals effectively used by each transaction type differ, even within the same OBI channel.

This observation allows the OBI channels to be reconsidered in terms of these four transaction categories and, as a result, to define payloads of different sizes for each category, improving the efficiency of transaction transmission over the link, as discussed in Section 4.2.4. The role of the two-bit tag field then becomes to identify the transaction type represented by the payload body. Since this tag determines how the remaining payload must be interpreted, it serves as the header flit of the payload and is therefore always sent first when a payload is issued to the next node. The reason for this ordering is further discussed in Section 4.4.1 and Section 4.4.2.

4.2.3 Routing and Payload Classification

Every payload that arrives at a node is classified by comparing its source and destination identifiers against the node's own identifier. This classification drives all subsequent routing decisions:

- **Transit**: the destination is some other node. The payload must continue around the ring toward its destination and is not consumed locally. This may happen in the data link layer or the protocol layer.
- **Incoming request**: the destination is this node and the tag denotes a request. The node must replay the request on its local address space and eventually return a response.
- **Incoming response**: the destination is this node and the tag denotes a response. The node must deliver it to the host as the answer to one of its own outstanding requests.

- **Loop-back**: the source is this node. A payload that has travelled the entire ring and returned to its originator indicates that it could not be delivered, and is handled as an error condition (Section 4.3.5).

Classifying on the source first is what makes loop detection robust: a payload that circles the ring without being claimed is recognized by the node that emitted it, rather than circulating indefinitely.

4.2.4 Dynamic Payload Sizing

Because the four transaction classes differ greatly in size, transmitting every payload at the size of the largest one wastes transmission cycles. A write response, for instance, conveys little more than an acknowledgement, whereas a write request must carry an address and a full data word.

Concretely, for a minimal OBI configuration with 32-bit address, 32-bit read and write data word, 3-bit transaction identifier, and up to 16 nodes network (i.e. 4-bit source and destination ID), the resulting payload head would be 10 bits and the body for each payload type would be:

Payload Type	Size	Fraction of Biggest Payload	Phits
Write Req.	81	100%	6
Read Req.	49	$66.\overline{6}\%$	4
Read Resp.	46	50%	3
Write Resp.	14	$16.\overline{6}\%$	1

Table 1: Example of payload sizes

Sending all of them at the largest size would mean transmitting 67 redundant zero-padded bits for every write response, a heavy penalty when the link is only a handful of wires wide.

Dynamic payload sizing removes this waste by transmitting only as many phits that are required to reconstruct the corresponding payload type. The size of each type is rounded up to a whole number of phits, and the receiver, having read the tag from the first phit, knows exactly how many cycles to expect for that type. The maximum payload size still bounds the buffering dimensions and the reassembly structures, but the common, small transactions now occupy the transmission wires for a fraction of the time, which directly improves the effective data rate.

4.2.5 Request and Response Life Cycle

Figure 5 traces a complete transaction through the ring. The originating node accepts a request from its host on the serial link OBI subordinate, forms a request payload, and injects it in the ring. The payload transits intermediate nodes hop by hop until it reaches the destination, which recognizes it as an incoming request, replays it by issuing the request to the host through the serial link's OBI manager, and forms a response payload addressed back to the original source. The response then transits back around the ring to the originator, which recognizes it as an incoming response and returns it to its host.

The two halves of the journey (request outbound, response returning) together traverse the full ring, so the round-trip latency grows with the size of the network.

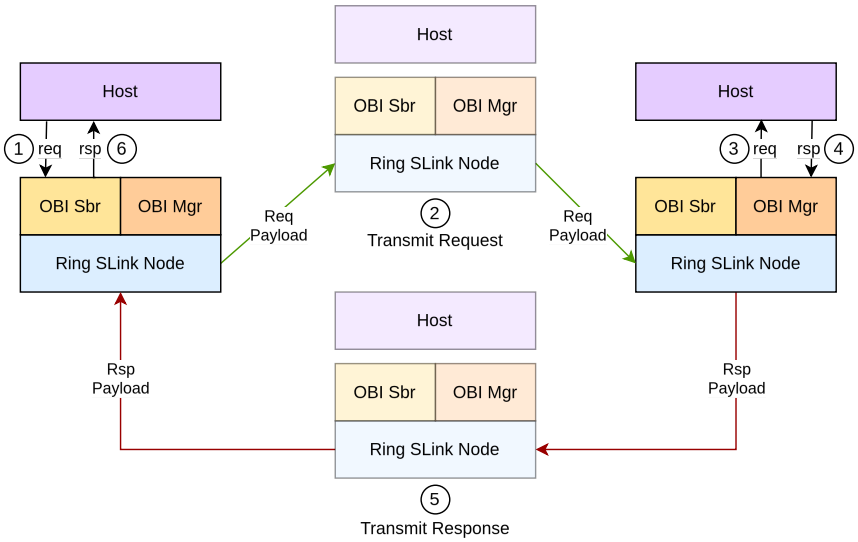


Figure 5: Request and response flow through a four-node ring: request transits to its destination, is served locally, and the response transits back to the originating node.

4.3 Protocol Layer

The protocol layer is the interface between the host's OBI bus and the link, and it is the part of the IP that was most thoroughly redesigned, since it had to be migrated from AXI to OBI and extended with the ring's networking logic. Its responsibilities are to serialize and deserialize transactions between bus channels and payload bodies, to decide which payload is admitted to the link next, to replay incoming requests on the local address space and route the resulting responses back to their origin, and to deliver responses to the host in the order the host expects. Figure 6 gives an overview of these datapaths.

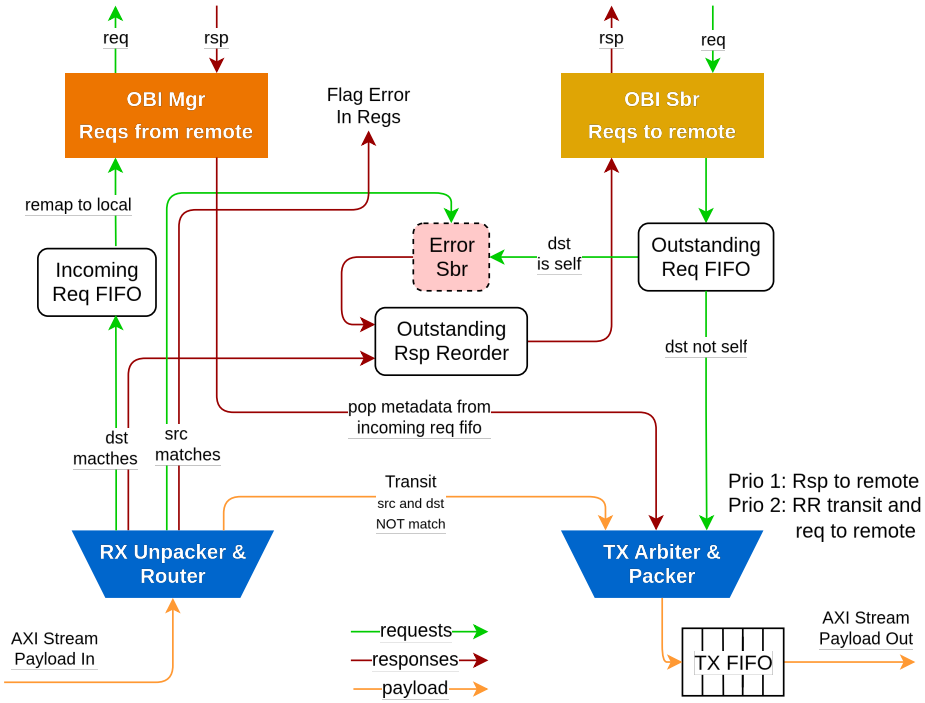


Figure 6: Datapaths of the protocol layer: serialisation and deserialisation of OBI transactions, the transmit arbiter, remote-request servicing, and the response reorder buffer.

4.3.1 Arbitration of Outgoing Payloads

At any moment three distinct streams may compete for access to the link layer, all of which must eventually be staged in the node's transmit buffer: outgoing requests that the host issues toward a remote node, responses that this node produces for requests it has serviced on behalf of a remote node, and transit payloads merely passing through. A transmit arbiter resolves the contention with a two-level priority policy:

- **Highest priority** is given to outgoing responses. As explained next, an OBI response phase cannot be back-pressured, so a response that becomes available must be captured immediately or it is lost; it therefore preempts the other streams.
- **Equal priority**, resolved by a round-robin rotation, is shared between outgoing host requests and transit payloads. The round-robin alternation guarantees that neither the local host nor the forwarded ring traffic can starve the other.

4.3.2 Backpressure and the Unbuffered-Response Problem

The transmit buffer is finite, and if the downstream node drains it more slowly than it fills, it can become full. For requests and transit payloads this is benign: they simply stall, since both their sources can be back-pressured. Responses are the difficulty. The OBI response phase has no handshake that the node can withhold, so when the local address space produces a response it must be accepted in the same cycle. If the transmit buffer were full at that instant, the response would be dropped.

The design solves this by reserving capacity in advance. The node bounds the number of remote requests it admits for local service, and for each admitted request it conceptually reserves a slot in the transmit buffer for the response that request will eventually generate. Requests and transit payloads are then only allowed to enqueue while free buffer space exceeds the number of slots reserved for outstanding responses. In effect, the space available to non-response traffic is deliberately reduced so that an unbufferable response is always guaranteed a home. This coupling between admitted requests and reserved response slots is the central flow-control invariant of the protocol layer.

4.3.3 Servicing Remote Requests

When a node receives a request addressed to it, it strips the node-identifier bits from the address to recover the local offset and replays the access on its manager port to the local address space, subject to the admission limit above. To be able to route the eventual response back to the requester, the node records, in an in-flight request tracker, the source identifier of the requester, the read/write kind, and the transaction identifier, in the order the requests were accepted. When the local address space produces a response, the oldest tracked entry is consumed, the response is wrapped into a response payload addressed to the recorded source, and it wins arbitration into the transmit buffer by virtue of its top priority. The first-in-first-out ordering of the tracker matches responses to requesters without any per-transaction search.

4.3.4 In-Order Response Delivery: The Reorder Buffer

A host expects the answers to its own outstanding requests to behave coherently, but responses may come back from the ring in a different order than the requests were issued, for example when successive requests target nodes at different distances. The protocol layer therefore contains a response reorder buffer that restores order while adding no latency to in-order responses.

The buffer is organized as a circular structure with a head and a tail pointer. Each time the host issues a request into the ring, a slot is allocated at the tail and the tail advances. To keep the transported identifier as narrow as possible, the host-supplied transaction identifier is not sent over the ring; instead the allocated slot index is injected as the payload's identifier, and the original host identifier is saved alongside the slot. This is why the transported identifier need only be wide enough to enumerate the outstanding slots, which keeps payloads small.

When a response returns, its identifier is exactly the slot index that was injected, so it indexes directly into the buffer. The saved host identifier is restored onto the response, and the response is placed in its slot. Delivery to the host happens strictly from the head: whenever the slot at the head holds a completed response it is handed to the host and the head advances. The buffer is zero latency because a response that arrives for the slot currently at the head is forwarded combinationally to the host in the same cycle, bypassing the storage entirely; only out-of-order responses are actually written into their slots to wait their turn. The buffer reports full when the slot at the tail is still awaiting completion, which is the signal that throttles the host.

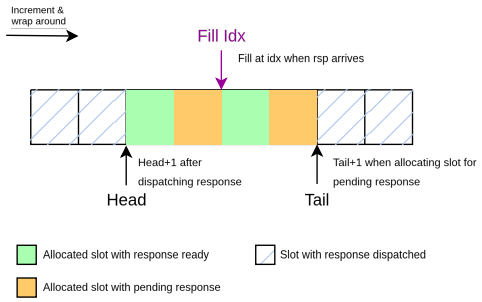


Figure 7: Response reorder buffer. Out-of-order responses wait in indexed slots; a response for the head slot is forwarded to the host combinationally.

4.3.5 Loop-Back Detection and Self-Addressed Requests

Two corner cases must be handled gracefully so that an undeliverable payload cannot circulate forever. The first is a *looped request*: a request payload that returns to its own source has visited every node without any of them claiming it, which means its destination is not present on the ring. The original node detects this from the source identifier, and completes the corresponding outstanding entry in its reorder buffer with an error response rather than waiting indefinitely. The second is a *looped response*: a response that returns to the node that produced it could not be delivered to its requester; nothing useful can be done with it, so it is discarded and a sticky error status bit is raised to inform software.

A related case is a request a host issues to its own node identifier. Injecting it would send it all the way around the ring only to come back as a loop, so instead the protocol layer short-circuits it: the request is admitted, a reorder slot is allocated, and an error response is scheduled locally for that slot. Self-addressed accesses thus complete promptly with an error instead of consuming ring bandwidth.

4.4 Data Link Layer

As its name suggests, the data link layer forms the interface between the physical and protocol layers. On the receive path, it converts the serialized phits arriving from one or more physical channels into the wide, parallel payload format expected by the protocol layer. On the transmit path, it performs the inverse operation, serializing the parallel payloads provided by the protocol layer for transmission over the physical links.

The data link layer is also responsible for credit-based flow control, a function that was moved from the protocol layer as part of the redesign. To accommodate the requirements of the ring network, it was extended with support for dynamic payload handling and a low-latency forwarding path for transit traffic, allowing packets destined for other nodes to traverse the network with minimal buffering and processing overhead.

Analysis of the original design revealed that the intermediate flow-control FIFO between the data link and protocol layers was not required for correct operation. Consequently, it was removed from the redesigned architecture. Completed payloads are now forwarded

directly to the stream registers or, in the case of transit traffic, directly to the next node. This simplification reduces buffering requirements and latency while preserving the functionality of the original design.

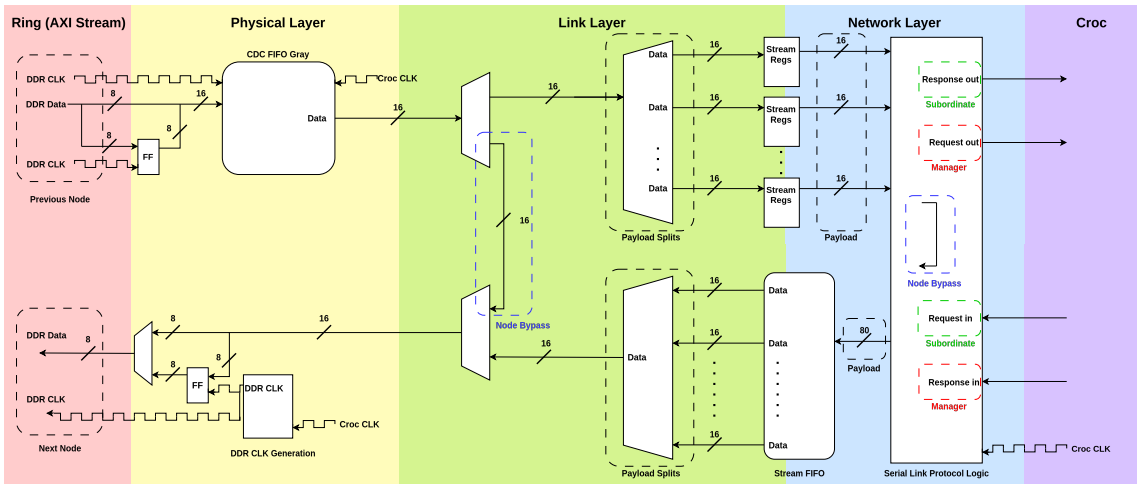


Figure 8: Block Diagram of the adapted Serial Link's RX and TX datapaths.

4.4.1 Serialization and Reassembly with Dynamic Sizes

When the data link layer receives data from the physical layer, it can either bypass the protocol layer, as described in Section 4.4.2, or reassemble the incoming serial stream into a parallel payload representation. The latter operation is illustrated in Figure 8. Because payloads are transferred over multiple cycles, the reassembly process uses a counter to track the number of received phits.

To support dynamic payload sizes, the original design was extended to inspect the transaction type as soon as the first phit arrives. The payload header determines the expected payload size and therefore the number of phits that must be collected. This information is used to initialize the reassembly counter and to immediately clear any stream-register locations that will not be used by the current payload. As a result, the reconstructed payload is always fully defined, regardless of its size. Once all required phits have been received, the stream registers signal the protocol layer that a complete payload is available. The protocol layer then consumes the payload and initiates the next reassembly cycle.

Serialization on the transmit path follows the reverse process. The data link layer receives a parallel payload from the protocol layer and serializes it into phits for transmission. As on the receive side, the payload header is used to determine the payload size and initialize a transmission counter. The data link layer transmits only the words that belong to the payload and stops once the required number of transmission cycles has been reached, rather than transmitting the zero-padded extension fields.

4.4.2 Protocol-Layer Bypass

A transit payload does not need to pass to the protocol layer by the node it passes through. Passing it up into the protocol layer to be reassembled and then re-serialized would cost several cycles per hop, and even more for larger payloads. The data link layer

therefore implements a bypass that forwards incoming phits directly from the receive side to the transmit side, visible as the shortcut in Figure 8. As with reassembly, the tag in the first phit tells the bypass how many cycles the transit payload spans, so it forwards exactly that many and then returns to its idle state. The bypass is only triggered when the data link layer determines from the first phit that the destination is not the current node, while the node is not sending out data from the protocol layer to the next node.

When the node is itself busy injecting traffic, transit payloads instead take the regular path through the protocol layer, where they are arbitrated against the local traffic. Consequently the speed-up is largest when the ring is lightly loaded. Which optimizes for a more realistic case, since not every node is always sending out data from the protocol layer.

4.4.3 Credit-Based Flow Control and the Credit Clock

The ring keeps the credit principle of the original design but expresses it in a way that suits the unidirectional topology. A credit corresponds to one free entry in the predecessor-to-successor direction's receive buffer, counted in phits rather than whole payloads, and a node is permitted to drive a phit only while it holds at least one credit. Its credit count is initialized to the depth of the downstream receive buffer and is decremented on every phit it sends.

Credits are returned out of band, over a dedicated credit-return clock line that runs counter to the data direction. Each time the receiving node drains a phit from its receive buffer it owes one credit to its predecessor, which it discharges by sending a single pulse on that clock line through a clock gate. The predecessor recovers each returned credit by crossing these pulses into its own clock domain through a CDC FIFO Gray and incrementing its credit count per pulse. Encoding a credit as nothing more than a clock edge keeps the return path to a single wire and avoids a separate back-channel payload format, at the cost of carrying no information beyond the credit count itself.

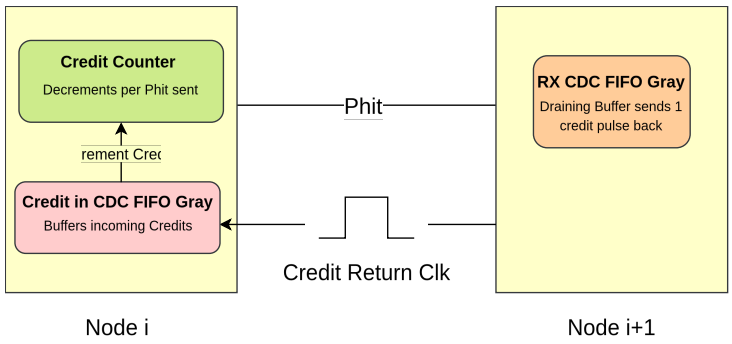


Figure 9: Credit-based flow control across one hop. Each word drained by the receiver returns one credit to the sender as a single pulse on the counter-rotating credit-return clock.

The minimum credit count needed to sustain continuous sending at clock division four can be derived as follows. Let

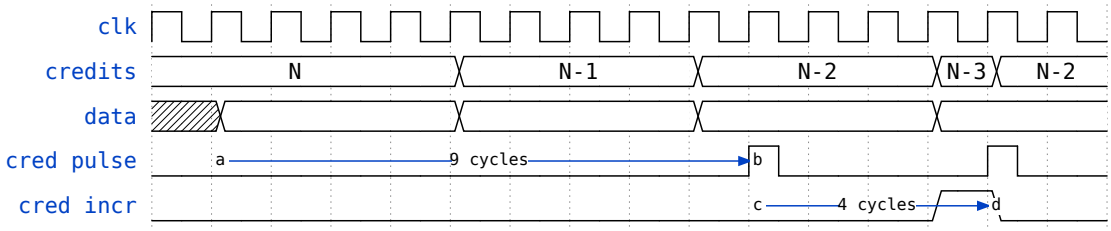
- $C_{\min}$ be the minimum credit count required to never run out of credits,

- $T_s = 4$ be the number of SoC cycles needed to send one phit at clock division four, and
- $T_r = 13$ be the number of SoC cycles needed for the first credit to come back (point a to d in the waveform diagram).

While sending continuously, the node consumes one credit every T_s cycles. The first credit returns only after T_r cycles, but every subsequent credit follows after just T_s cycles, because the returns are pipelined. The critical window is therefore the initial latency before the first credit arrives: the node must hold enough credits to keep sending throughout it. This gives

$$C_{\min} \geq \frac{T_r}{T_s} = \frac{13}{4} = 3.25 \quad \Rightarrow \quad C_{\min} = 4.$$

Since the credit count is bounded by the depth of the RX CDC FIFO Gray buffer, this sets the minimum usable depth at clock division four.



Listing 1: Latency of returning first credit

4.4.4 Multi-Channel Operation

The mechanism that distributes the phits of a payload across several link channels is inherited from the original design and remains present, because the surrounding rework left it largely untouched. It is possible to use the ring with more than one channel and it retains those functionalities in the simulation of the adapted design.

4.5 Physical Layer

The physical layer is reused with only minimal modifications and handles all off-chip signalling except for the credit signals. On the transmit side, it drives the lanes of each channel and forwards a source-synchronous clock generated by a programmable divider. The divider ratio, as well as the positions of the clock's rising and falling edges, can be configured at run time, allowing software to control the link frequency, phase, and duty cycle to optimally center clock transitions within the data eye.

When DDR is enabled, each phit is split into two halves that are transmitted on alternating clock phases. On the receive side, the incoming lanes are sampled on both clock edges to reconstruct the complete phit. The recovered data is then transferred through a CDC into the node's system clock domain.

4.6 Configuration and Control

Software configures and observes a node through a small register file, accessed over a dedicated OBI configuration port. The original IP connected these registers over APB the ring variant moves them to OBI so that the entire IP presents a single, consistent bus protocol to the host and so that nodes can also reach one another's configuration registers over the ring. This avoids the need for a separate peripheral interconnect alongside the OBI data bus. The most fundamental control fields establish a node's place in the ring and its link timing: the node identifier that the routing logic compares against, a send stop control that halts transmission and forwarding (and is set after reset so software can complete bring-up before the node participates in the ring), and the divider, phase, and duty settings of the forwarded clock. A sticky status bit records the looped-response error condition described in Section 4.3.5. The complete set of software-visible registers is listed in Table 2.

Register / Field	Offset	Access	Reset	Description
General Control and Status				
`ctrl`	`0x000`	rw	`0x1`	**stop_send** — Halts the transmitter and forwarding of payloads to network layer. Reset to `1`, so sending is stopped until software clears it.
`error`	`0x004`	rw1c	`0x0`	**looped_rsp** — Set when a response is looped back to the originating node. Write `0` to clear.
`node_id`	`0x008`	rw	`0x0`	ID of this Serial Link IP within the ring network (Log2MaxNodeIds bits wide).
`flow_control_fifo_clear`	`0x024`	w	`0x0`	Clears the flow control FIFO of the RX datapath.
TX PHY Clock Configuration [1]				
`tx_phy_clk_div`	`0x300`[1]	rw	`0x8`	Clock division factor for the forwarded TX PHY clock.
`tx_phy_clk_start`	`0x400`[1]	rw	`0x2`	Position of the rising edge of the divided, phase-shifted TX clock (duty-cycle and phase control). Reset is `0x4` when DDR is disabled.
`tx_phy_clk_end`	`0x500`[1]	rw	`0x6`	Position of the falling edge of the divided, phase-shifted TX clock. Reset is `0x0` when DDR is disabled.

[1] The listed reset values for the TX PHY clock registers assume DDR is enabled (`EnDdr = 1`); the DDR-disabled values are given in the respective descriptions.

Register / Field	Offset	Access	Reset	Description
Raw Mode				
`raw_mode_en`	`0x00c`	w	`0x0`	Enables raw mode, forwarding incoming data to output for testing purposes.
`raw_mode_in_ch_sel`	`0x014`	w	`0x0`	Selects the channel to read received data from in raw mode.
`raw_mode_in_data_valid`	`0x100`[2]	r	—	Mask marking valid data in the RX FIFOs during raw mode (hardware-written).
`raw_mode_in_data`	`0x010`	r	—	Data received on the selected channel in raw mode (NumBits wide, hardware-written).
`raw_mode_out_ch_mask`	`0x200`[2]	w	`0x0`	Selects the output channels in raw mode; all ones broadcasts on every channel.
`raw_mode_out_data_fifo`	`0x018`	w	`0x0`	Data pushed into the raw mode output FIFO.

Register / Field	Offset	Access	Reset	Description
`raw_mode_out_data_fifo_ctrl`	`0x01c`	w	—	**clear** — Clears the raw mode TX FIFO.
		r	`0x0`	**fill_state** [15:8] — Number of entries currently held in the raw mode TX FIFO.
		r	`0x0`	**is_full** [31] — Set when the FIFO is full; further writes are ignored until space frees up.
`raw_mode_out_en`	`0x020`	rw	`0x0`	Enables transmission of the data currently held in the output FIFO.
Channel Allocator – TX [3]				
`channel_alloc_tx_cfg`	`0x600`	rw	`0x1`	**bypass_en** — Bypasses the TX channel allocator.
		rw	`0x1`	**auto_flush_en** — Enables the auto-flush feature on the TX side.
		rw	`0x2`	**auto_flush_count** [15:8] — Cycles to wait before auto-flushing (sending) packets.
`channel_alloc_tx_ch_en`	`0x700`[2]	rw	`0x1`	Per-channel enable mask for the TX side.
`channel_alloc_tx_ctrl`	`0x604`	w	—	**clear** — Software-clears the TX side of the channel allocator.
		w	—	**flush** — Transmits any remaining data on the TX side.

[2] The register is instantiated once per channel (an array of length `NumChannels`); the offset shown is the base address.

Register / Field	Offset	Access	Reset	Description
Channel Allocator – RX [3]				
`channel_alloc_rx_cfg`	`0x608`	rw	`0x1`	**bypass_en** — Bypasses the RX channel allocator.
		rw	`0x1`	**auto_flush_en** — Enables the auto-flush feature on the RX side.
		rw	`0x2`	**auto_flush_count** [15:8] — Cycles to wait before synchronizing on partial packets.
		rw	`0x1`	**sync_en** [16] — Enables the synchronization barrier between channels; must be disabled in raw mode.
`channel_alloc_rx_ch_en`	`0x800`[2]	rw	`0x1`	Per-channel enable mask for the RX side.
`channel_alloc_rx_ctrl`	`0x60c`	w	—	**clear** — Software-clears the RX side of the channel allocator.

Table 2: Software-visible configuration registers of the Serial Link IP. All registers are 32 bit wide.

[3] The channel allocator registers are read/write only when channel allocation is enabled (`EnChAlloc`, i.e. `NumChannels > 1`); otherwise they are read-only.

Access codes: **rw** read/write, **r** read-only, **w** write-only. A reset value of "—" denotes a register with no defined reset value, i.e. a write-only strobe or a field driven by hardware.

4.7 Removal of the Wrapper

The original IP was enclosed in a wrapper that allowed the entire module to be shut down: it gracefully terminated all outstanding bus transactions so that the host clock to the link could be gated, kept the configuration registers clocked so software retained

control, and provided a software reset that reinitialized everything except those registers. This made sense for a point-to-point bridge, where quiescing one endpoint merely stops traffic between the two SoCs.

In a ring this shutdown is no longer meaningful. Deactivating a single node does not isolate it harmlessly; it severs the ring and cuts communication among all remaining nodes, because every node is a forwarding element on the shared path. Since the connection isolation existed solely to enable that full shutdown and a clean software reset is not feasible in the ring without risking the loss of in-flight payloads, the wrapper no longer serves a purpose and has been removed. Shutting down and resetting a node in a ring context would instead require coordinated, ring-aware mechanisms, which are beyond the scope of this project.

5

Integration into Croc SoC

5.1 Functionality and Testing within Croc SoC

The modified Serial Link IP has been integrated into the user domain of the Croc SoC to verify its functionality within a SoC and to demonstrate a complete end-to-end integration. The Serial Link IP was configured with one link channel and 8 lanes, and the pins of the Croc SoC were updated accordingly. In its original configuration, the Croc user domain has only one manager and one subordinate connection to the OBI crossbar. This has not been changed for the integration, which means the configuration register subordinate and the ring subordinate share a demultiplexer. A manager within the Croc SoC therefore cannot access both simultaneously.

The address mapping of the Croc SoC has been modified so that addresses with the upper 4 Most Significant Bit (MSB) set are routed to the ring, as shown in Table 3.

Subordinate	Start Address	End Address
`Ring Serial Link Configuration Registers`	`0x0FFFF000`	`0x0FFFF804`
`User Domain Error Subordinate`	`0x0FFFF805`	`0x0FFFFFFF`
`Ring Serial Link Subordinate`	`0x10000000`	`0xFFFFFFFF`

,

Table 3: Address mapping of the Croc SoC user domain with the integrated Ring Serial Link IP

Node ID	Address Range	Scope
-	`0x00000000 - 0x0FFFFFFF`	Local Address Space
1	`0x10000000 - 0x1FFFFFFF`	Global Address Space
2	`0x20000000 - 0x2FFFFFFF`	
3	`0x30000000 - 0x3FFFFFFF`	
4	`0x40000000 - 0x4FFFFFFF`	
…	…	
12	`0xC0000000 - 0xCFFFFFFF`	
13	`0xD0000000 - 0xDFFFFFFF`	
14	`0xE0000000 - 0xEFFFFFFF`	
15	`0xF0000000 - 0xFFFFFFFF`	

Table 4: Network address space layout of the Croc ring network

To complete the integration, the start addresses of other Croc SoC components, including the bootrom and the Static Random-Access Memory (SRAM) banks, were adjusted accordingly. To make the IP accessible from software, C drivers were written that allow a user to read and write data to and from the ring and to modify the configuration registers.

To demonstrate the functionality of the Ring Serial Link IP, a test program was written in C. The program sets the node ID and increases the transaction clock from $f_{\text{Croc}}/8$ to $f_{\text{Croc}}/4$ while retaining a 90° phase shift. It then activates the node and begins issuing write requests to the other nodes in the ring. Once the responses are received, it issues read requests to verify that the data read back matches what was previously written. Each write and read request targets a distinct address within SRAM bank 1 of the destination node, determined by the source ID, destination ID, and current test index. The program uses preprocessor directives to allow the node ID, the number of tests, and the number of nodes in the ring to be configured at compile time.

The test program was loaded onto multiple Croc instances in a testbench and all tests completed successfully. Signal analysis in GTKWave confirmed that the integration is functional and that the ring operates as intended.

5.2 Backend Flow

To demonstrate that the modified Serial Link IP does not only work with ideal and unrealistic timing simulations but also with more realistic constraints, the Croc SoC with the integrated ring Serial Link IP has been pushed through the backend flow. Using the 130nm BiCMOS open-source Process Design Kit (PDK) [1], the design has been successfully placed and routed.

Figure 10 shows how much area the ring Serial Link IP takes up on the chip compared to the rest of the Croc SoC. The largest part of the design is clearly the physical layer, because of the size of the RX CDC FIFO Gray buffer. The protocol layer is the second

largest part and is not even a third of the RX CDC FIFO Gray buffer in size. The RX CDC FIFO Gray buffer grows significantly when its depth is increased, or when its width — that is, the number of lanes — increases. In addition, each new channel adds its own physical layer, so the number of RX CDC FIFO Gray buffers grows as well. This has to be kept in mind when increasing the number of lanes and channels, since both make the design take up significantly more area. It should also be noted that increasing the depth of the CDC FIFO Gray buffer increases the number of credits, which in turn increases the size of the credit-receive CDC FIFO Gray buffer. The credit-receive CDC FIFO Gray buffer is already not small, and it only grows with more credits.

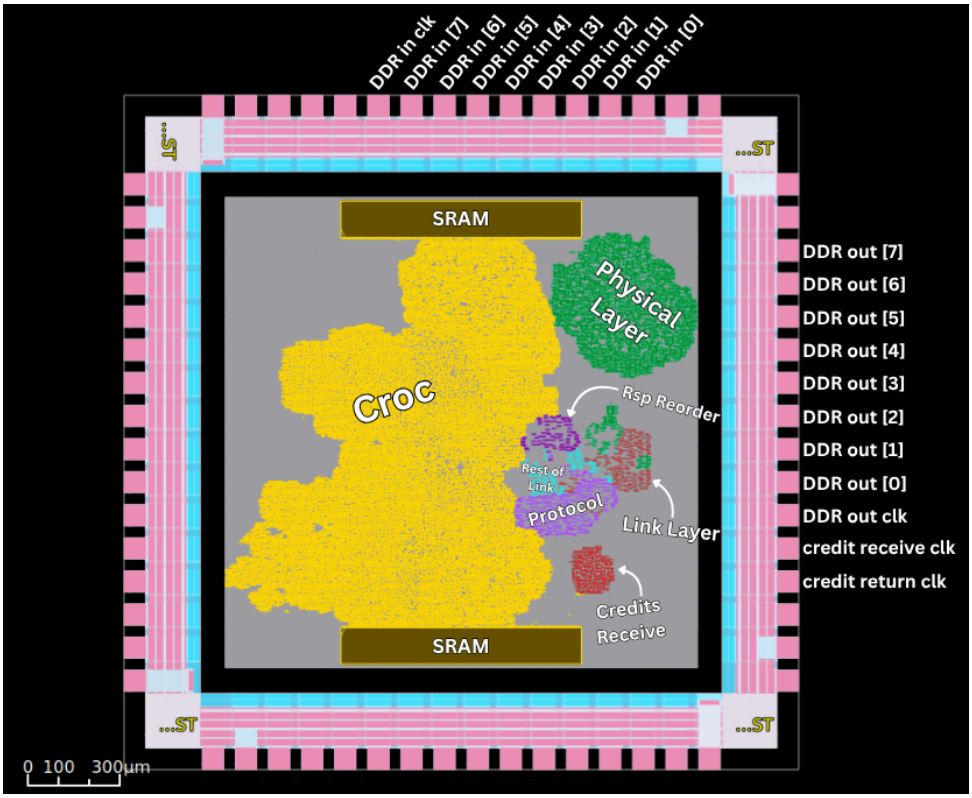


Figure 10: Layout of Croc chip with ring serial link integrated.

# Channels	1	TX FIFO Depth	3
# Lanes	8	RX CDC FIFO Depth	4
DDR	enabled	Max Outstanding	2

Table 5: Ring Serial Link configuration for the integrated IP

5.2.1 Design Area For Multiple Channels

#Channels	Area kGE
1	25.3
2	30.5
3	30.4
4	36.8
5	37.2

Table 6: Post-synthesis area results for multiple channels using IHP [1] 130nm process.

Table 6 shows how much area, in kilo Gate Equivalent (GE), multiple channels take up. Notably, the total area decreases when going from 2 to 3 channels. This is because the size of the RX CDC FIFO Gray buffers depends on the number of payload splits needed to send the biggest payload in the ring. Since adding channels also adds lanes, it can reduce the number of payload splits for the biggest payload. This is what happens when going from 2 to 3 channels: the RX CDC FIFO Gray buffers added to each channel are smaller in total than the two larger buffers of the 2-channel design.

6

Simulation Results

To evaluate the performance of our design we have conducted a series of tests with various configurations and benchmarked the average latency to complete a transaction, the effective throughput of each byte of OBI data read/written, as well as the aggregated ring throughput of all bits sent including payload headers and source/destination information. In the following sections, we will define as *goodput* the effective OBI data throughput, and as *throughput* the aggregated throughput of each bit sent.

6.1 Experimental Setup

The tests were conducted by implementing a testbench that connected a variable amount of ring serial link nodes together and hooked each node IP to an OBI manager driver and an OBI subordinate driver provided by PULP-Platform [5]. All simulations were conducted using Questa Sim-64 (vsim 10.7b_1 Simulator 2018.07). All nodes run with a base clock frequency of 100MHz.

Five network scenarios were considered for the experiments:

- **All-to-One**: All nodes except one send simultaneously read/write requests to the node that is not sending.
- **All-to-all**: All nodes send read/write requests in loop to all nodes in increasing distance order.
- **One-to-all**: Only one node sends read/write requests in loop to all nodes in increasing distance order.
- **All-to-next**: All nodes simultaneously send read/write requests to their next neighbour.
- **All-to-prev**: All nodes simultaneously send read/write requests to their previous neighbour.

Unless specified otherwise the parameters used for the simulations are the following:

# Channels	1	TX FIFO Depth	3
# Lanes	8	RX CDC FIFO Depth	4
# Nodes	16	Max Outstanding Requests	2
DDR	enabled	Max Inflight Requests	2
Prot. Bypass	enabled	Clk Division	8
Dyn. Payload Sizes	enabled	TXNs per Node	2048

6.2 Performance

6.2.1 Scaling with respect to ring size

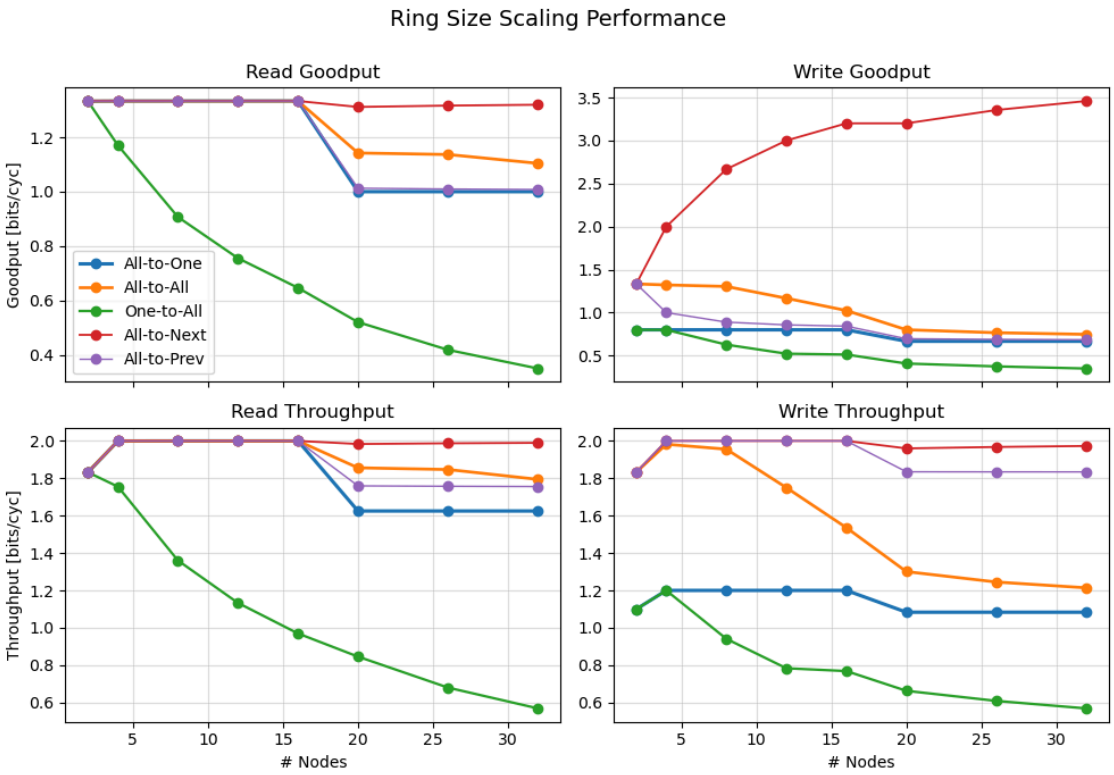


Figure 11: Effect of ring size on throughput and goodput

The read goodput stays roughly constant across most of the loads. The exception is the one-to-all load, where the read goodput drops the larger the ring gets. The reason is straightforward: in this load only one node sends requests, and it has to wait for each response to come back. The more nodes the response has to pass through, the longer this takes. Since only two transactions can be in flight at a time, this latency cannot be hidden, and the read throughput drops for the same reason.

From 20 nodes onwards, the goodput of the all-to-one, all-to-previous, all-to-all and all-to-next loads is also reduced. The reason is that 16 nodes is the largest ring whose IDs still fit in 4 bits; from 20 nodes onwards, the destination and source IDs each need one extra bit. These two extra bits make the read request one cycle longer to send, which directly reduces the goodput.

How much each load drops depends on how many hops a read request travels. In all-to-next a read request travels only one hop, so the drop is small. In all-to-previous it travels every hop except the last, so the drop is much larger.

This does not yet explain why the all-to-one load drops as sharply as the all-to-previous one. The reason there is different: the all-to-one load already has a bottleneck at the node just before the target, where the requests from all the other nodes converge. The two extra bits make every payload at this bottleneck about 25 percent longer in cycles,

which reduces the bottleneck's throughput and therefore the load's goodput by roughly the same fraction.

It is also worth noting the two-node case. With only two nodes, the source and destination IDs need just one bit each, so the read and write throughput is lower than at four nodes where each payload simply contains fewer bits. The goodput is unaffected, however, because these ID bits do not change the number of cycles needed to send one payload.

The write goodput behaves as expected. For the all-to-next load, adding more nodes increases the goodput, because more nodes mean more write requests and therefore more data flowing through the ring. This gain is not cancelled out by the added latency, for two reasons: the write requests only travel one hop, so they do not fill the ring with much data, and the write responses stay at one cycle per hop, so the extra latency from more nodes remains small.

6.2.2 Benefits from dynamic payload sizing

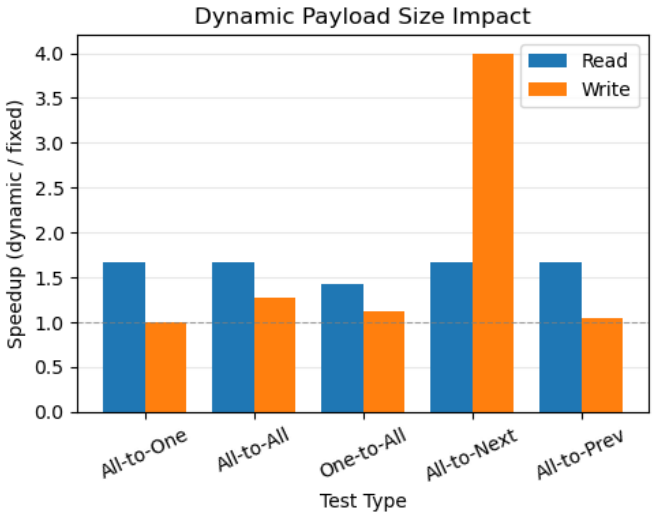


Figure 12: Impact of dynamic payload sizes on performance

Dynamic payload sizing shrinks every payload type except the biggest one: the write request. Figure 12 shows the effect on each load.

Read requests and responses become significantly faster in every load except the one-to-all one. In one-to-all the speedup is limited because only two read transactions can be in flight at a time; allowing more outstanding read transactions would unlock further goodput.

Write requests are unaffected, since they are the biggest payload. Write responses, on the other hand, shrink dramatically: a dynamic write response takes only one cycle to send. Yet this does not help every load equally.

For the all-to-one load, shorter write responses give no speedup. The bottleneck in this load sits on the request path, at the node just before the target. The write responses leave the target in the opposite direction and travel the rest of the way around the ring,

away from the congested section. Shortening them therefore does nothing to relieve the bottleneck.

The all-to-all load also gains very little. The write request payloads dominate the traffic, so the shortened write responses save very little per transaction.

The one-to-all load also gains little. The initiating node does send some write requests to its immediate neighbour, and on their own these would behave like the all-to-next case and give a good speedup. But the same node also sends write requests to nodes further away, and those long-distance requests share the ring with the fast write responses and slow them down. The load thus bottlenecks itself with too many outstanding transactions.

The all-to-next load achieves the biggest speedup. Here every write request travels just one hop, so the rest of the ring is filled with the now-very-short write responses. Short, fast payloads dominating the ring is exactly what good goodput requires.

This observation suggests a useful pattern for real workloads: instead of having a single node write to many distant nodes, traffic should be organized so that each node passes data to its successor, which then passes it on to its own successor, and so on around the ring. The speedup over the alternative is large, so shaping the ring load to look like the all-to-next pattern is well worth the effort.

The all-to-previous load sees only a small speedup. Its write responses each travel only one hop, so shortening them frees up very little wire time.

6.2.3 Effects of the protocol layer bypass

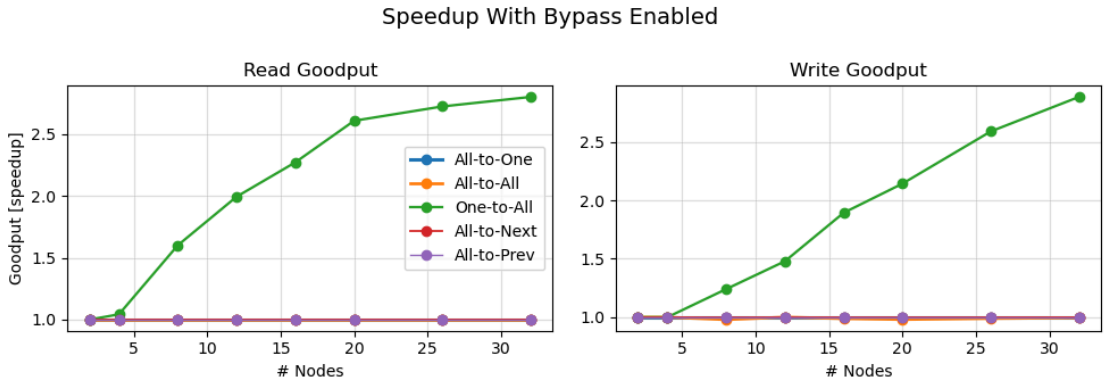


Figure 13: Effects of the protocol layer bypass on performance

The protocol-layer bypass gives a significant speedup whenever a payload passes through a node that is not sending any data of its own. The more idle nodes a ring contains, the more often the bypass is triggered and the more cycles are saved. The one-to-all load has exactly this property, since most nodes are not sending, so its speedup from the bypass is large and grows with the ring size.

Although one-to-all is one of the worst-performing loads overall, it is the one that shows the bypass at its best. The all-to-next load performs better in total, but it is hard to reach in a real task, since not every node always has data to send to its neighbour. A more

realistic pattern is one node starting a transfer and the next forwarding it onward. While this happens, the remaining nodes are idle, which is exactly the situation the bypass speeds up.

In the other loads every node is busy at all times, so the bypass is rarely triggered.

6.2.4 Scaling with respect to number of physical lanes

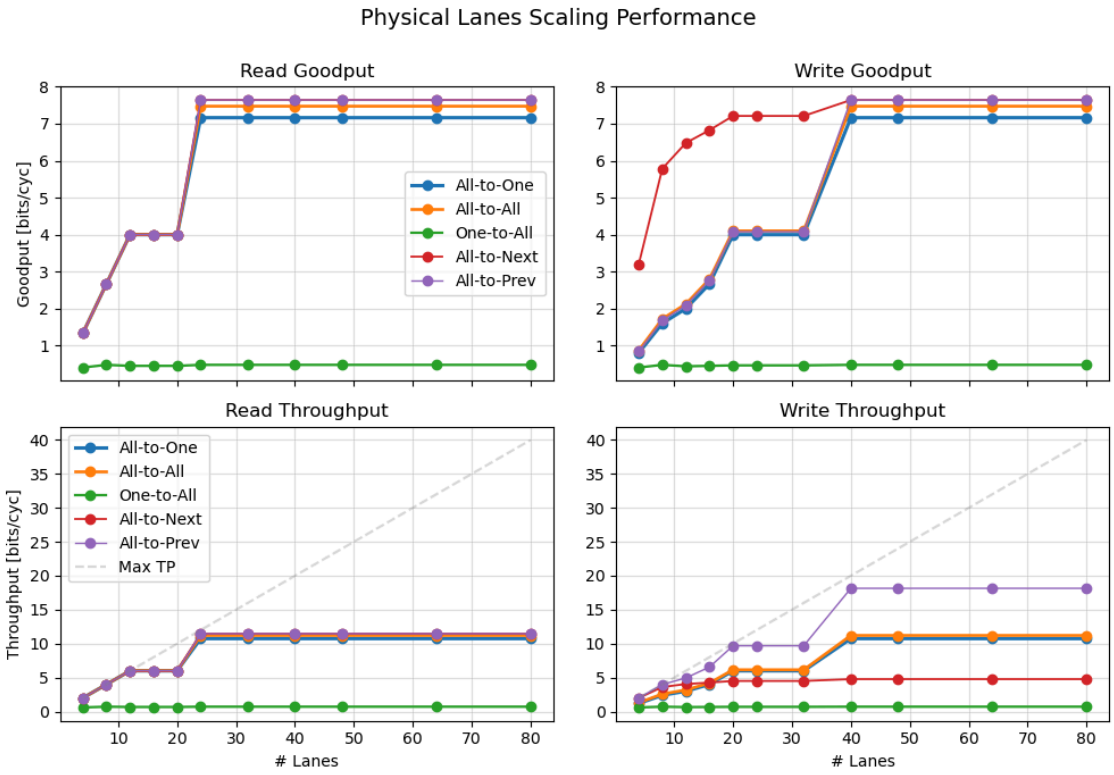


Figure 14: Effect of number of lanes on throughput and goodput

Adding more lanes lets the ring send more bits per cycle. By itself, however, this does not improve the goodput. What matters is whether the extra width is enough to reduce the number of cycles it takes to send a complete payload of any type. Only then does the goodput actually grow.

One can observe this quite well because increasing the amount of lanes does not inherently increase the throughput nor the goodput.

All ring loads except for the one-to-all arrive at a similar goodput ceiling. The reason for why they differ is due to the fact that every request or response that arrives at its destination introduces a delay of one or two cycles respectively. If these delay cycles do not happen at the same time, it slows down the other payloads. Hence the all-to-all load has a performance decrease and which is even more pronounced in the all-to-one ring load, where the processing of the requests only happen at one node which slows down the other payloads.

The one-to-all load is again the exception. Its goodput and throughput barely grow by adding lanes and channels due to the limiting factor being the round-trip latency: the single sender has to wait for the responses to its requests before it can issue more. With

only two transactions allowed in flight, a higher per-cycle bandwidth does not help while the latency stays the same.

6.2.5 Scaling with respect to RX CDC FIFO Gray Depth

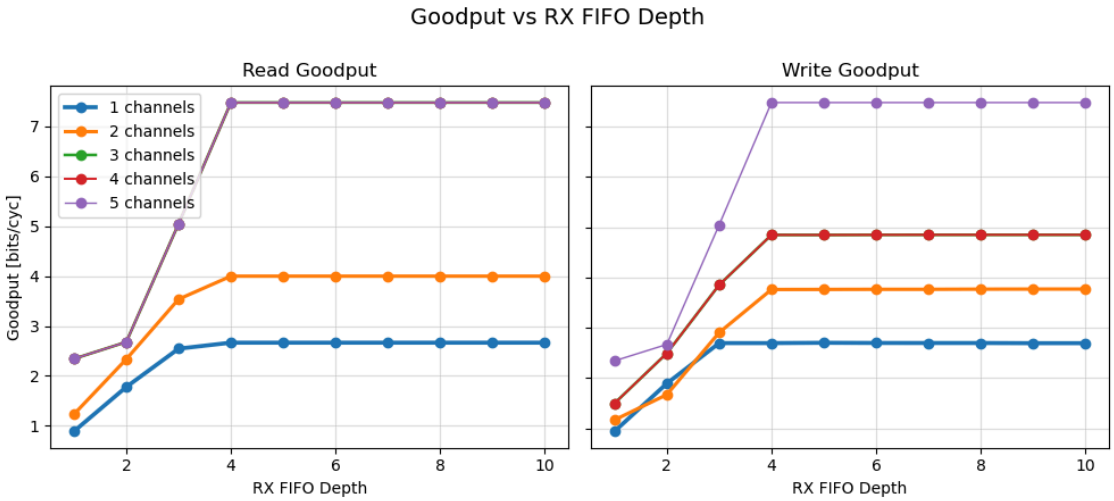


Figure 15: Effect of FIFO Depth on goodput

As calculated in Section 4.4.3, the optimal depth for the CDC FIFO Gray buffer is four, and this simulation confirms that theoretical analysis. The number of channels has no influence on the optimal depth, because every CDC FIFO Gray buffer is filled during a transaction even when the extra entries hold only zeroes. Whether the ring uses one channel or several therefore makes no difference to credit starvation: the depth affects all channels equally.

6.2.6 Scaling with respect to number of outstanding transactions

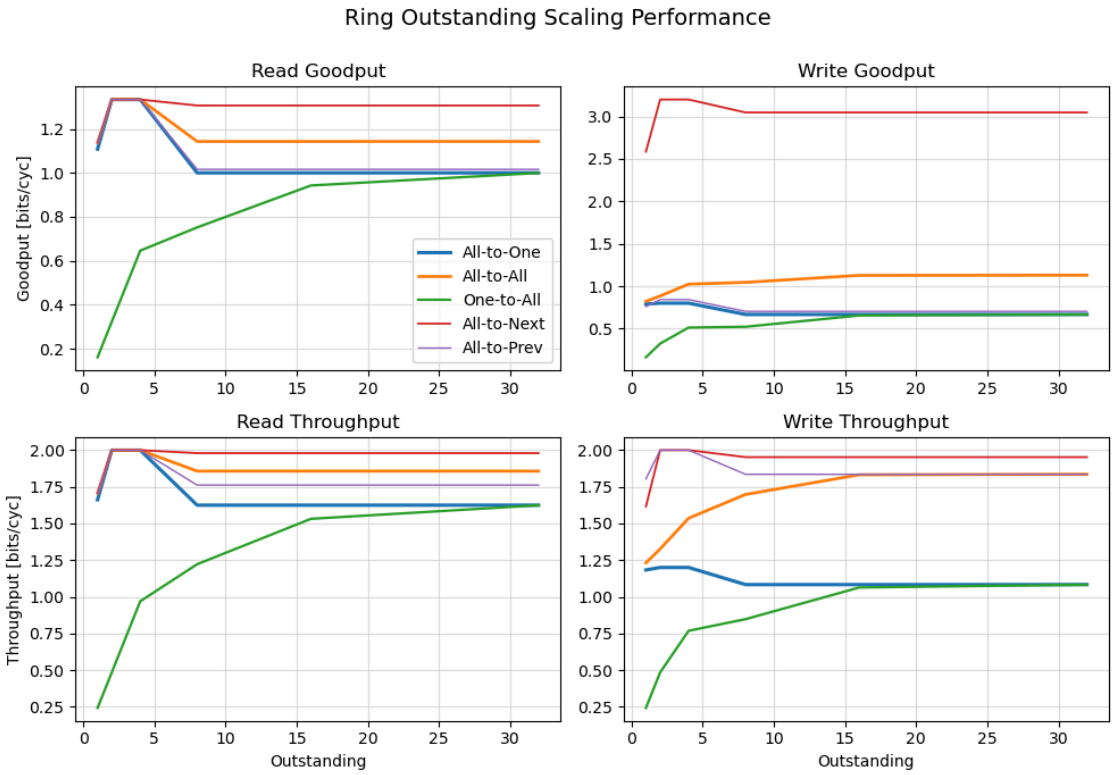


Figure 16: Effect of number of outstanding transactions on throughput and goodput

Allowing more concurrent transactions has two opposing effects. On one hand, more outstanding transactions push the ring closer to its maximum capacity. On the other hand, more outstanding transactions need a wider transaction ID field, which might make a payload one or more cycles longer to send, depending on the payload type.

Starting from a single outstanding transaction the first effect dominates, and the goodput grows. For most loads, however, the gain is quickly cancelled by the longer payloads. Beyond a certain point the goodput begins to drop again because of that.

The exception is once more the one-to-all load, whose goodput keeps growing throughout. Here the bottleneck is the round-trip latency of a single transaction, not the wire bandwidth. Letting the sender keep more transactions in flight directly attacks that bottleneck and continues to pay off.

7

Conclusion

This project extended the point-to-point Serial Link IP into a node of a unidirectional ring interconnect, allowing more than two SoCs to communicate over a simple, low-pin-count topology. The host-facing bus was migrated from AXI to OBI, and the upper layers were extended with the networking logic a ring requires: routing based on the node bits of the address, classification of payloads as transit, incoming, or looped, in-order response delivery through a zero-latency reorder buffer, and graceful handling of undeliverable payloads.

Two optimizations keep each hop cheap. Dynamic payload sizing sends every transaction type at its own size instead of padding all of them to the largest one, and the protocol-layer bypass forwards transit traffic without reassembling it whenever the node is not itself sending. Flow control kept the credit principle of the original design but was reshaped for the unidirectional ring, returning credits as single pulses on a counter-rotating clock line.

The resulting IP was integrated into the user domain of the Croc SoC, where a C test program running on multiple instances exercised reads and writes across the ring and completed successfully. The design was then pushed through the backend flow and successfully placed and routed in the 130nm IHP technology, showing that it works under realistic timing constraints and not only in idealized simulation. Finally, its performance was characterized across five ring-load scenarios, which confirmed the benefits of dynamic payload sizing and the bypass and revealed how the design scales with ring size, lane count, and the number of outstanding transactions.

7.1 Future Work

There are numerous directions for further work. A complete error-handling mechanism for faulty payloads in the ring could be added — for example, for the case where a response payload arrives at a SoC even though it never issued the corresponding request. In such a case one could shut down the ring, reset the ring Serial Link, and start the ring up again. Other, more error-specific solutions could also be explored. Additionally, a proper start-up procedure for the ring could be introduced with a new payload type. The RX CDC FIFO Gray buffer could be analyzed and modified to reduce area and to determine the best depth for a given lane size. The deadlock-prevention mechanism could likewise be improved. Finally, the ring could be fitted with an address look-up table so that addresses can be remapped; this would replace deriving the node ID from

the MSBs and would allow for more nodes while giving each a smaller ring-accessible address space.

A

Assignment Description

ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

Semester Project at the Department of
Information Technology and Electrical Engineering

Spring Semester 2026

Aegerter Fabian
Muela Llorenc

Improving Serial Link and Enabling a Ring Bus Topology

February 24, 2026

Advisers: Pu Deng (IIS), ETZ J 78, pudeng@iis.ee.ethz.ch
Tim Fischer (IIS), OAT U21 fischeti@iis.ee.ethz.ch
Handout: February 23, 2026
Due: June 1, 2026

1 Project description

Modern SoC platforms increasingly rely on chip-to-chip links to attach external devices, connect chiplets, or build multi-node systems. For this purpose, PULP provides an open and synthesizable serial link IP [3]. It is a scalable, source-synchronous DDR (double data rate) link with an AXI interface and a layered structure (protocol/data-link/physical), making it suitable for both high-bandwidth and low-latency use cases.

In its current form and typical deployments, a serial link is often used as a point-to-point building block. To scale to larger systems, an attractive next step is a ring topology: multiple nodes connected in a closed chain, enabling multi-hop communication with predictable wiring and incremental scalability [4]. This project explores how to further develop the existing peripheral towards such a ring-bus capable design, while optionally pushing performance improvements (or combining both).

This project will be conducted in conjunction with the croc SoC [1] in the VLIS2 course. The students will first modify the network layer of the Serial Link to ensure compatibility with the OBI protocol [2]. Subsequently, they will extend the design by integrating ring network functionality. After completing the architectural modifications, the design will be implemented through the full backend flow, culminating in the generation of the final chip layout in GDS format. Finally, the students will evaluate the resulting design by analyzing its power consumption, performance, and area (PPA) through simulation.

2 Project Goals

The project aims to achieve the following milestones (M):

1. **M1:** Familiarize with the Croc SoC, and the Serial Link. Modify the Serial Link to work with OBI protocol (2W).
2. **M2:** Design the architecture of the Ring Network Router. Clearly specify the interfaces and modules' logic under different working condition (1W).
3. **M3A:** Implement the sender part of the ring network router (5W).
4. **M3B:** This is in parallel with M3A. Implement the receiver part of the ring network router (5W).
5. **M4:** Push the design through the backend flow and generate GDS (2W).
6. **M5:** Perform the simulations, evaluate the design's PPA. (1W).
7. **M6:** Optional: Add dead lock prevention logic.
8. **M7:** Optional: Add automatic initialization logic.
9. **M8:** Final report writing and presentation (3W).

2.1 Project Plan

Within the first month of the project, the student will be asked to prepare a project plan. This plan should identify the tasks to be performed during the project and individual tasks' deadlines. The prepared plan will be discussed with the supervisors during the first week's meeting. Note that the project plan should be updated constantly depending on the project's status.

2.2 Meetings

Weekly meetings will be held between the student and the supervisors. These meetings' exact time and location will be determined during the project's first week. These meetings will be used to evaluate the status and progress of the project and to advise and support the student. Besides these regular meetings, additional meetings can also be organized to address urgent issues.

2.3 Report

Documentation is an important and often overlooked aspect of engineers. One final report has to be completed within this project. The final report will be written in English. Any form of word processing software is allowed for writing the reports, nevertheless the use of LaTeX with Tgif[1] or any other vector drawing software (for block diagrams) is strongly encouraged by the IIS staff.

Final Report The final report has to be presented at the end of the project, and a digital copy needs to be handed in. Note that this task description is part of your report and has to be attached to your final report.

2.4 Presentation

There will be a presentation (15 min presentation and 5 min Q&A) at the end of this project to present your results to a wider audience. The exact date will be determined towards the end of the work.

References

[1] pulp-platform. *Croc: A PULP SoC for education with a complete physical design flow.* `https://github.com/pulp-platform/croc`. Accessed: 2026-02-20. 2026.

[2] pulp-platform. *OBI: SystemVerilog synthesizable interconnect IPs for the OBI v1.6 standard.* `https://github.com/pulp-platform/obi`. Accessed: 2026-02-20. 2026.

[1] See: `http://bourbon.usc.edu:8001/tgif/index.html` and `http://www.dz.ee.ethz.ch/en/information/how-to/drawing-schematics.html`.

[3] pulp-platform. $serial_link : A simple, scalable, source-synchronous, all-digital DDR link$. https://github.com/pulp-platform/serial_link. Accessed: 2026-02-20. 2026.

[4] *Ring network*. Wikipedia, The Free Encyclopedia. Last edited 23 January 2026, accessed 20 February 2026. 2026. URL: https://en.wikipedia.org/wiki/Ring_network.

Zürich, February 24, 2026 Prof. Dr. Luca Benini

4

B

Declaration of Originality

ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Improving Serial Link and Enabling a Ring Bus Topology

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**
Hueka Haussmann Aegerter	Lorenz Fabian

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**
Zürich, 01.06.2026 Zürich, 01.06.26	

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).

Bibliography

[1] I. P. Authors, "IHP Open Source PDK." [Online]. Available: https://ihp-open-pdk-docs.readthedocs.io/en/latest/

[2] O. Group, "OpenBus Interface Specifications." [Online]. Available: https://github.com/openhwgroup/obi/blob/072d9173c1f2d79471d6f2a10eae59ee387d4c6f/OBI-v1.6.0.pdf

[3] Pulp-Platform, "Serial Link." [Online]. Available: https://github.com/pulp-platform/serial_link

[4] P. Platform, "PULP Platform." [Online]. Available: https://pulp-platform.org/

[5] Pulp-Platform, "Obi/src/test/obi_test.Sv at main · pulp-platform/obi." [Online]. Available: https://github.com/pulp-platform/obi/blob/main/src/test/obi_test.sv

[6] F. Aegerter and L. Muela, "Ring Serial Link." [Online]. Available: https://github.com/faegerter/Ring-Serial-Link

[7] F. Aegerter and L. Muela, "Croc with Ring Serial Link Integrated." [Online]. Available: https://github.com/faegerter/croc-with-ring-bus-IP